

Delhivery Hub Reliability & SLA Simulation

Does fleet size or SLA design drive on-time delivery?

Jnanadyuti Patra

September 2026

Code & data: <https://github.com/Jnanadyuti-Patra/delhivery-hub-reliability-sim>

Live dashboard: <https://jnanadyuti-patra.github.io/delhivery-hub-reliability-sim/>

1 Problem Statement

Delhivery, one of India’s largest integrated logistics providers, dispatches shipments from hub to hub across the country. Like any delivery network, it sometimes misses its delivery promise. There are two structurally different explanations for a missed service-level agreement (SLA), and each implies a different fix:

1. **A capacity problem.** If shipments are delayed because they are queued waiting for an available vehicle, the fix is operational: add more vehicles to the dispatch fleet.
2. **A promise-design problem.** If vehicles are freely available but individual trip times are inherently variable (traffic, weather, road conditions, distance), then adding vehicles does nothing — the fleet was never the bottleneck. The fix here is to set a more realistic delivery promise, or to change *how* shipments are routed, not how many vehicles carry them.

This report documents a discrete-event simulation built on Delhivery’s own real operational data to distinguish between these two explanations for one real hub, and to turn the answer into a concrete, quantified recommendation.

2 Data

The dataset is Delhivery’s public operational release: **144,867 real GPS/scan records**, covering trips across Indian cities and states between 12 September and 6 October 2018. Row count and field structure were verified against the documented public dataset before use.

Each shipment is tracked through a `trip_uuid`, moving through one or more `source_center` → `destination_center` legs. For every leg, the data records:

- **route_type:** **Carting** (smaller, typically local pickup/delivery runs) or **FTL** — Full Truck Load (one truck dedicated to a single shipment, typically longer routes).
- **actual_time:** the real, observed transit time for that leg (minutes).
- **osrm_time:** the transit time predicted by OSRM, an open-source routing engine — effectively the “map estimate,” analogous to a Google Maps ETA.
- **osrm_distance:** the routing engine’s planned distance for the leg (km).

A shipment is scanned at multiple checkpoints as it moves, which means a single leg between two centers can appear as several rows with increasing cumulative `actual_time/osrm_time` values. The cleaning step below collapses these into one real observation per leg.

3 Methodology

No machine learning model was trained for this study. The problem being asked — “*if we changed the fleet size or the routing policy, what would happen?*” — is a counterfactual, causal question about a system’s operating mechanics, not a pattern-recognition question about historical correlations. A predictive ML model (e.g. a regression predicting delivery time from features) would only tell us what *already happened* under the fleet size and policies actually used historically; it cannot tell us what would happen under a fleet size or policy that was never tried. **Discrete-event simulation (DES)** was chosen instead, because it lets us

directly rebuild the mechanism — vehicles, queues, dispatch — and then run it under hypothetical conditions that never occurred in the real data.

3.1 Step 1 — Cleaning the Real Data

src/01_clean_data.py collapses the repeated checkpoint scans for each (trip_uuid, source_center, destination_center) leg down to one final observation, using the *maximum* cumulative actual_time / osrm_time recorded for that leg (the last checkpoint before the shipment moved on):

```
1 grp = df.groupby(["trip_uuid", "source_center", "destination_center"],
2                 as_index=False).agg(
3     route_type=("route_type", "first"),
4     actual_time=("actual_time", "max"),
5     osrm_time=("osrm_time", "max"),
6     osrm_distance=("osrm_distance", "max"),
7     ...
8 )
9 grp["delay_ratio"] = grp["actual_time"] / grp["osrm_time"]
```

This produced **26,003 clean trip legs** across **1,487 source hubs** nationally, each with a real **delay_ratio** — how many times longer the trip actually took versus what the routing engine predicted.

3.2 Step 2 — The Simulation Model

src/02_simulate_hub_policy.py models outbound dispatch at **Gurgaon_Bilaspur_HB**, the busiest source hub in the dataset (1,062 legs), restricted to its **Medium-distance tier** (77–299 km) — the one tier where both Carting and FTL are genuinely used side by side in the real data, making it the actual decision point a dispatcher faces.

The simulation is built with **SimPy**, a Python discrete-event simulation library. Two features of the real world are modeled explicitly:

- **A limited vehicle pool**, represented as a `simpy.Resource`. A new shipment must acquire a vehicle before it can depart; if all vehicles are busy, it queues, exactly as it would at a real dispatch yard.
- **Bootstrapped real timing**, not a fitted theoretical distribution. Instead of assuming transit times follow (say) a Normal or Exponential curve, every simulated trip's (actual_time, osrm_time) pair is drawn *with replacement* directly from the real historical legs of that route type. This preserves the true, messy relationship between planned and actual time, including its long right tail.

```
1 def dispatch_proc(dispatch_id):
2     ready_time = env.now
3     rt = "Carting" if rng.random() < hist_carting_share else "FTL" \
4         if policy == "Historical" else policy
5     actual_time, osrm_time = draw_leg(rng, rt) # real bootstrapped pair
6     with fleet.request() as req:
7         yield req # wait for a free vehicle
8         wait = env.now - ready_time
9         yield env.timeout(actual_time) # occupy it for the real trip time
10        total_time = env.now - ready_time
11        on_time = total_time <= SLA_BUFFER * osrm_time
12        log.append((rt, wait, actual_time, osrm_time, total_time, on_time))
```

Dispatches themselves arrive following the real historical *interarrival gaps* observed at this hub and tier — also bootstrapped, so simulated traffic is as bursty (or as calm) as the real hub actually was. The simulation is run for 150 independent replications per scenario (each a simulated 30-day month) to average out sampling noise.

3.3 Step 3 — Experiment 1: Is It a Fleet-Size Problem?

With the SLA promise fixed at $2\times$ the OSRM-planned time (close to the $\sim 2.1\text{--}2.5\times$ average delay observed nationally), the outbound fleet size was swept from 2 to 24 vehicles, under three routing policies: all-Carting, all-FTL, and the real historical 55%/45% Carting/FTL mix. For every combination, on-time rate and average dispatch wait time were recorded.

3.4 Step 4 — Experiment 2: What SLA Can Each Policy Actually Deliver?

`src/03_sla_buffer_sweep.py` follows directly from Experiment 1’s finding (Section 4). With fleet size fixed at 12 vehicles — a level at which queueing wait is already effectively zero — the *SLA buffer itself* was swept from $1.5\times$ to $5\times$ the planned time, to find the tightest delivery promise each routing policy can hit 90% of the time.

4 Results

4.1 The raw pattern: real deliveries run well past the plan

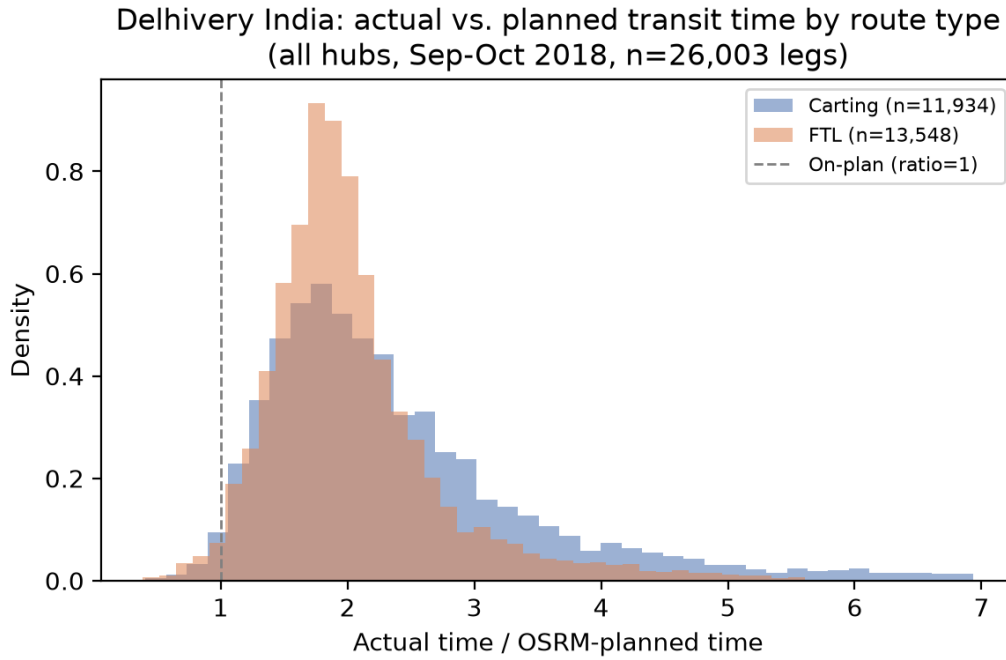


Figure 1: Actual-to-planned transit time ratio, all 26,003 legs nationally. A ratio of 1.0 means the delivery took exactly as long as the routing engine predicted. Carting averages $2.51\times$ the plan; FTL averages $2.15\times$.

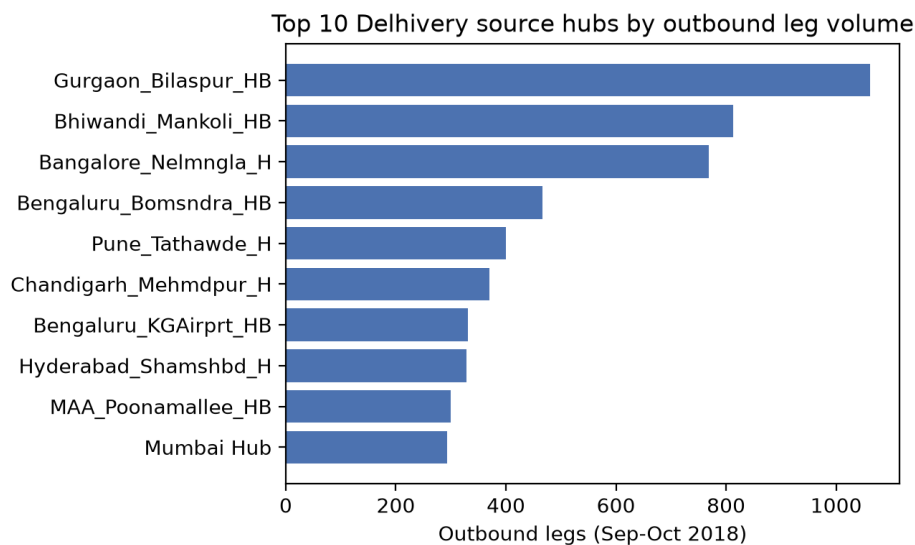


Figure 2: Gurgaon_Bilaspur_HB is the single busiest outbound hub in the dataset, making it the natural hub to model in depth.

4.2 Experiment 1: fleet size is not the bottleneck

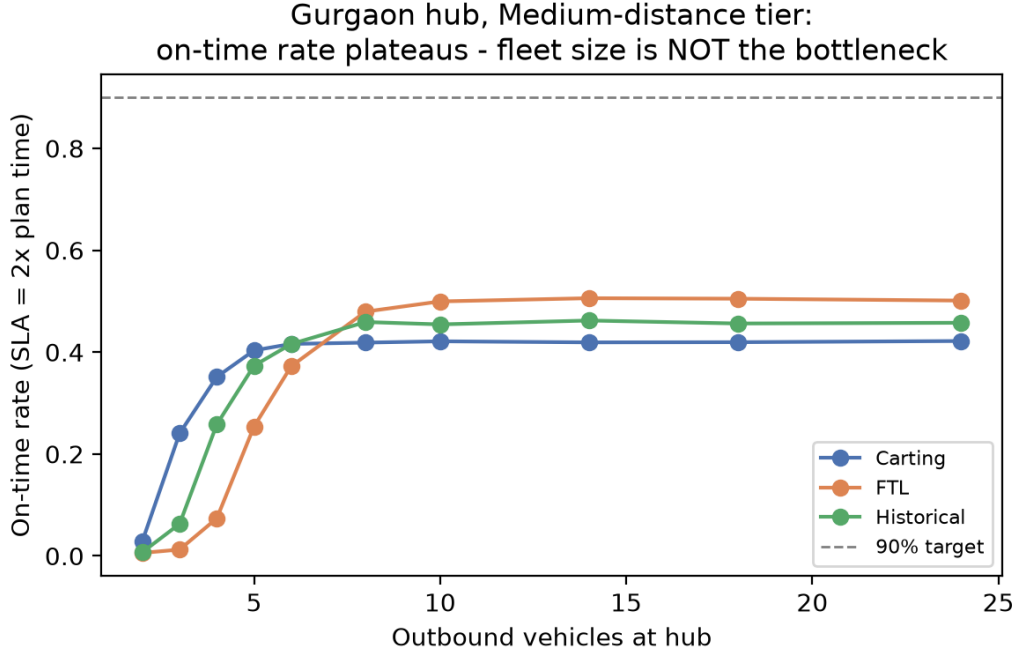


Figure 3: On-time rate vs. outbound fleet size, at a fixed $2\times$ SLA. By ~ 8 vehicles, average dispatch wait time is already near zero (the queue has emptied) — yet on-time rate plateaus at only 42–50%. Adding more vehicles beyond this point changes nothing.

Finding: this rules out Explanation 1 from Section 1. The hub is not capacity-constrained at this SLA target; the ceiling on on-time performance comes from the inherent variability of individual trip times, not from vehicles sitting idle in a queue.

4.3 Experiment 2: the real lever is routing policy and SLA design

Policy	Min. buffer for 90% on-time	Avg. delay ratio (national)	Historical share
FTL	$3.0\times$ plan time	$2.15\times$	45%
Historical mix	$3.5\times$ plan time	—	—
Carting	$3.5\times$ plan time	$2.51\times$	55%

Table 1: SLA buffer required to hit a 90% on-time rate, by routing policy, Gurgaon hub, Medium-distance tier.

5 Recommendation

For Medium-distance corridors out of the Gurgaon hub, routing shipments via **FTL** instead of Carting (or the current historical mix) lets the hub promise a **tighter, shorter delivery window** ($3.0\times$ the routing engine’s planned time, versus $3.5\times$) while holding the same 90% reliability target. This is a zero-capital-expenditure recommendation — no new vehicles required — driven purely by which routing policy is used for this tier.

6 Limitations

- The dataset carries no fare or freight-cost field. Vehicle-hours consumed (not currency) was used as the operational-efficiency proxy throughout; a real per-trip cost model would let this recommendation be stated in rupees saved, not just buffer-time tightened.
- The analysis is scoped to one hub (Gurgaon_Bilaspur_HB) and one distance tier (Medium, 77–299 km), the tier with the clearest real-world overlap between Carting and FTL usage. Other hubs or tiers may show different dynamics.

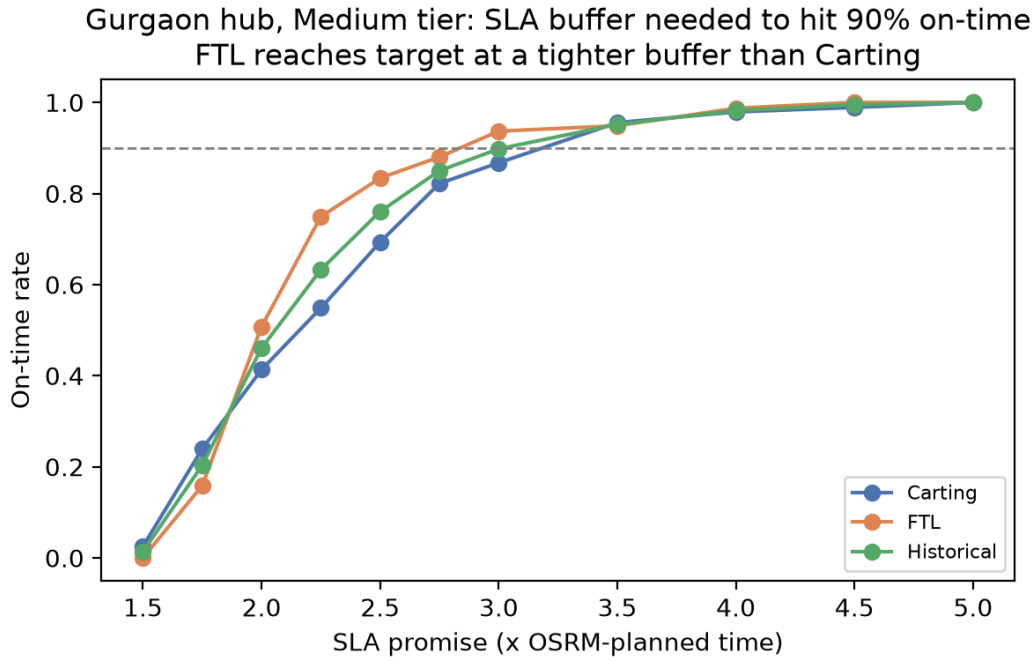


Figure 4: On-time rate vs. SLA buffer (fleet size fixed at 12, congestion-free). FTL reaches the 90% target at a tighter buffer than Carting or the historical mix.

- The dataset spans roughly three weeks (12 Sep – 6 Oct 2018); seasonal effects (festival traffic, monsoon disruption) are not represented.

7 Reproducibility

The full pipeline — data cleaning, both simulation experiments, and the chart-generation script — is published with documentation at: <https://github.com/Jnanadyuti-Patra/delhivery-hub-reliability-sim>. An interactive results dashboard is live at: <https://jnanadyuti-patra.github.io/delhivery-hub-reliability-sim/>.